Honors Section Additional Information

Welcome to the Honors section of CSCE 430. This is to explain the difference between how the honors section of the class and the regular sections of the class will operate.

The lecture and the lab portions of the class will run as usual, in conjunction with the rest of the class.

The weekly problem set assignments will be slightly different for you. Through the semester, there will be 14-16 weekly problem sets assigned. In 10 of those weeks, one of the problems will be designated as a non-honors problem, meaning it is not assigned to you. These problems will be identified on the canvas (discussion area) each week. Typically, this will be one of the easy or mid-range difficulty problems in the set. Please note that while you may solve these for your own benefit, you will not be able to get credit for them as bonus/extra credit or for upsolving.

You will, instead, be required to create two problems yourself.

- Each of these problems will count as 5 solved problems for determining your grade. While generally a complete solution will receive all 5 points, you may receive less than 5 points if you do a poor job (e.g. insufficient test, poor writing, etc.), and the problem may not be accepted at all if you have not provided all parts.
- You must submit these problems to pass the class with more than a C. If you do not submit both problems, or if either of your problems is not accepted, your maximum grade in the class (regardless of all other items) will be a C.
- One problem is due Sunday, March 15 (i.e. end of Spring Break)
- A second problem is due Tuesday, April 28 (last day of class)

Please note two things that have "caught" students in the past:

- It takes many students some effort to get the Kattis Tools working and checking results. You should allow plenty of time for this, in advance, and may expect to ask for help from (and offer help to) other students. **It is up to you** to get these working and testing your code with them. It will not be acceptable to submit code that does not pass.
- The effort expected on each problem is the equivalent of what it would take for 5 "regular" problems. That is, you should expect to devote time to this more than what you'd typically spend on a full week of regular problems. Though the task may seem simple at first, many things are required, and doing a thorough job will take significant effort. Do not assume this is something you can knock out easily in even just one full day of work.

Here is what is required from you for your problem creation:

- You should create a problem that meets the requirements of the Kattis Problem Format. Detailed links to the format are below, but generally this means including:
    - A problem description (.tex file giving the problem description, input format, and output format)
    - One or more sample inputs, with corresponding outputs
    - Multiple "secret" inputs used to **thoroughly** test your problem, with corresponding outputs. You must include tests that explore all edge cases, the extremes of input,

typical inputs, and different patterns of input (when appropriate). Typically this will mean writing a program that can generate test cases.
    - At least one correct solution to the problem
    - An input verifier that can be used to verify that an input matches the requirements
    - A metadata file
    - If desired (these are NOT required), you may include:
        - More than one correct solution program
        - Examples of solutions that should return wrong answer, run-time error, or time-limit exceeded results (useful especially if you want to ensure only efficient implementations are accepted)
        - An output validator (necessary if the output could have multiple correct answers or formats)
        - Hint, Description, or Illustration files for the test data.
    - The problem should pass the Kattis verification tools.
- A short writeup (for me) that describes the problem design
    - I expect this will be ½ page to 1 page in length
    - You should identify what the goal of the problem is, in terms of what concept(s) you are wanting to illustrate/test with the problem. For instance, if the problem is meant to involve a shortest-path algorithm, say so. If it's meant to provide lots of corner cases, say that. If it's meant to just involve a lot of programming that's straightforward, say that.
    - You should give a very brief explanation of how to solve the problem. That is, give a description for what a person is expected to do so that someone reading the description who is a decent programmer could go ahead and solve the problem.
    - List anyone who helped you with the problem, and in what capacity (see below).
- The expectation is that these problems may be submitted to Kattis for possible use, either in future versions of this class, or for more general use. *If for some reason you do not wish to allow your problem(s) to be used in this way, please indicate so, clearly, in your writeup.*
- At least one of your two problems should be something more than just a direct, simple application of a given algorithm. That is, it should cause the person solving it to have to do more than just implement a straightforward algorithm on a particular input format. Some examples of how this might work:
    - Require someone to implement multiple algorithms together – e.g. combine a graph problem with a combinatorics problem. Or, transform one problem to another type then solve that.
    - Create a problem that, in addition to having a direct solution, has a significant number of special cases that must be dealt with (possibly individually).
    - Create a problem that requires an interesting and not-very-straightforward mapping from the problem description to an algorithm. That is, create a problem that on first reading might seem like one of type X, but actually requires a solution of type Y.
- At least one of your two problems should exhibit a time-constraint, where a poor choice of algorithm would lead to a Time Limit Exceeded result. For such problems, you should be sure that your problem submission includes a TLE program in the solution section, and that your test cases clearly separate good solutions from those that would give a TLE result
- Your submission should be zipped into a single file:
    - The file should contain the problem writeup and a directory following the Kattis format for the problem information.

    - This one .zip file should be uploaded to Canvas. Note that this is the only time in the class that Canvas will be used for submitting problems.
- To help ensure that you are on the right track, each person will need to meet individually with the instructor prior to the final problem deadline. *At this meeting, you should have a draft of the short writeup describing problem design (see above) available for me to review.* These meetings will be scheduled and held (possibly by zoom, possibly in person) over a 2-week period ending approximately 1.5 weeks before the problem deadline, specifically:
    - For problem 1: Feb. 23-March 6
    - For problem 2: April 13-24

Please note: You should write all the required parts of the problem yourself, including a correct solution and the majority of the test cases. However, if you wish to include additional solutions (e.g. of wrong answers, or of multiple acceptable answers), you can include solutions written by other people. You may also include test cases provided by other people. **You are welcome to get feedback from others**, have others "try" your problem on their own, etc. before submitting it. In fact, you are encouraged to do so – others will typically be able to identify and point out ambiguities or other issues. They might also be able to help identify cases you did not consider.

I want this to be a fun and interesting experience. I believe you will find that the process of coming up with new problems is both challenging and intellectually stimulating.

See information about the Problem formatting process at:
https://www.kattis.com/problem-package-format/
The Kattis problemtools repository (for verifying the problems) is available from github:
https://github.com/Kattis/problemtools
You may find it useful to use the BAPCtools to develop your problem:
https://github.com/RagnarGrootKoerkamp/BAPCtools